

Manual de instalacion - AHG CONSTRUFERRET POS

Guia practica para instalar, operar y mantener el sistema.

Version documentada: julio de 2026

AHG CONSTRUFERRET POS

1. Arquitectura recomendada

La instalacion productiva utiliza:

- **Vercel** para publicar el POS, el portal de clientes, la IA local y las rutas API.
- **Supabase PostgreSQL** para conservar productos, clientes, inventario, pre-facturas, facturas y tracking.
- **IMECF** para la integracion de comprobantes electronicos en ambiente de prueba.

El repositorio incluye `vercel.json`, `api/index.py`, `schema/postgres.sql` y `requirements.txt`. SQLite queda reservado para desarrollo local; no debe usarse como base compartida en Vercel porque sus escrituras no son persistentes entre ejecuciones serverless.

Hay dos formas soportadas:

| Forma | Uso | Base de datos | URL |

|---|---|---|---|

| Local | Desarrollo, pruebas y demostracion | SQLite en `data/ahg_demo.db` | `http://127.0.0.1:8765` |

| Vercel | Uso remoto multiusuario | Supabase PostgreSQL Pooler IPv4 | `https://ahg-construferret-pos.vercel.app` |

La instalacion local no reemplaza la productiva: cada una puede tener su propia base y sus propias variables. Para trabajar con los mismos datos desde cualquier lugar se debe usar la ruta Vercel + Supabase.

2. Requisitos

- Cuenta de GitHub con acceso al repositorio.
- Cuenta de Vercel con el proyecto `ahg-construferret-pos`.
- Proyecto de Supabase y su contraseña de base de datos.
- Python 3.12 o superior para ejecución local.
- Git y navegador moderno.
- Vercel CLI solo si se desea desplegar desde PowerShell.

3. Instalacion local

Clona el repositorio y entra en la carpeta:

```
git clone https://github.com/raulemanuel9697-dev/ahg-construferret-pos.git
cd ahg-construferret-pos
```

Instala dependencias:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Si PowerShell bloquea la activacion, ejecuta `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` y repite la activacion.

4. Ejecutar en SQLite local

Para desarrollo y demostraciones:

```
$env:PYTHONPATH="$PWD\src"  
$env:AHG_DEMO_MODE="1"  
$env:AHG_HOST="127.0.0.1"  
$env:AHG_PORT="8765"  
$env:AHG_AUTH_SECURE_COOKIE=""  
python -m ahg_pos.app
```

Direcciones locales:

- POS: <http://127.0.0.1:8765/>
- Login: <http://127.0.0.1:8765/login>
- Portal cliente: <http://127.0.0.1:8765/catalog>

SQLite crea data/ahg_demo.db y carga datos de demostracion cuando esta vacia. El acceso inicial es admin@ahg.local con Cambiar123!; cambia esa contraseña antes de entregar el sistema.

5. Clave maestra local

Las credenciales IMECF se cifran con AHG_CREDENTIAL_SECRET. Genera una clave aleatoria y dejala persistente:

```
$bytes = New-Object byte[] 48  
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()  
$rng.GetBytes($bytes)  
$rng.Dispose()  
$secret = [Convert]::ToBase64String($bytes)  
[Environment]::SetEnvironmentVariable("AHG_CREDENTIAL_SECRET", $secret, "User")  
$env:AHG_CREDENTIAL_SECRET = $secret
```

No publiques la clave y no la cambies despues de guardar credenciales. En Vercel esta clave debe configurarse como variable sensible independiente.

6. Configurar Supabase

1. Abre el proyecto de Supabase.
2. Entra a **Project Settings -> Database -> Connect**.
3. Selecciona **Session pooler** y copia la cadena PostgreSQL.
4. Usa el host `aws-0-<region>.pooler.supabase.com`, el usuario `postgres.<PROJECT_REF>` y el puerto de Session pooler.
5. Sustituye la contraseña en la cadena sin compartirla en GitHub.

El proyecto actual usa el ref mytxighyegsqyoqxkuzg. La variable final debe llamarse DATABASE_URL y comenzar con postgresql://. Se recomienda Session pooler porque Vercel necesita un endpoint IPv4 compatible con funciones serverless.

El esquema se crea automaticamente al iniciar el POS. La primera ejecucion crea tablas, referencias, usuario administrador y productos demo si la base esta vacia. Para una carga controlada, ejecuta el contenido de schema/postgres.sql desde el SQL Editor de Supabase antes de iniciar.

7. Enlazar Vercel

Instala Vercel CLI si no esta disponible:

```
npm.cmd install -g vercel  
vercel.cmd login
```

En la raíz del proyecto:

```
vercel.cmd link --project ahg-construferret-pos
```

El archivo `vercel.json` redirige las rutas web y API al handler Python `api/index.py`. Vercel detecta la clase `handler` y construye las dependencias desde `requirements.txt`.

8. Variables de Vercel

Configura las siguientes variables en **Production** y, si se necesita, también en **Preview**:

```
DATABASE_URL=<cadena Session pooler de Supabase>  
AHG_CREDENTIAL_SECRET=<secreto aleatorio largo>  
AHG_DEMO_MODE=0  
AHG_AUTH_SECURE_COOKIE=1  
IMECF_BASE_URL=https://ecf-platform-backend-50801509587.us-central1.run.app  
IMECF_COMPANY_ID=<id de la empresa IMECF>  
IMECF_ENABLED=1  
IMECF_API_KEY=<clave de prueba IMECF>
```

Las variables sensibles se agregan desde **Vercel -> Project -> Settings -> Environment Variables** o con `vercel env add`. Nunca guardes `DATABASE_URL`, `AHG_CREDENTIAL_SECRET` ni `IMECF_API_KEY` en `.env`, GitHub, capturas o documentos.

9. Desplegar a producción

Desde la raíz del repositorio:

```
vercel.cmd pull --yes --environment production  
vercel.cmd deploy --prod --yes
```

El dominio productivo actual es:

```
https://ahg-construferret-pos.vercel.app/
```

Rutas principales:

- `/login`: autenticacion administrativa.
- `/`: POS.
- `/catalog`: portal cliente para preorden y asesor IA local.
- `/api/public/products`: catalogo publico con paginacion.
- `/api/public/categories`: categorias disponibles.

10. Validacion posterior al despliegue

Comprueba:

1. La raíz redirige a `/login`.
2. El login administrativo abre el POS.
3. El catalogo muestra productos y categorias.

4. La paginacion cambia de pagina sin perder filtros.
5. El asesor IA responde usando el inventario de Supabase.
6. Una preorden aparece en **Pre-Facturas**.
7. El RNC se consulta cuando corresponde.
8. **Gestion Fiscal** muestra la empresa IMECF.
9. **Validar emisor** y **Probar conexion** responden correctamente.
10. Una factura de prueba conserva su e-NCF, estado y tracking.

Para revisar errores de despliegue:

```
vercel.cmd logs ahg-construferret-pos.vercel.app
```

11. Configurar IMECF desde el POS

1. Inicia sesion con rol admin.
2. Abre **Gestion Fiscal**.
3. Edita o crea la empresa.
4. Completa workspace, razon social, RNC, ambiente y URLs.
5. Pega la API key sin el prefijo x-api-key:.
6. Confirma la contraseña administrativa y guarda.
7. Pulsa **Validar emisor**.
8. Pulsa **Probar conexion**.
9. Pulsa **Activar** cuando las pruebas sean correctas.

Usa ambiente test durante la demostracion. No emitas comprobantes reales sin validar primero secuencias, RNC, ambiente y autorizacion fiscal.

12. Copias de seguridad y seguridad

- En Supabase configura copias y conserva el proyecto con acceso restringido.
- No compartas la contraseña de la base, la clave maestra ni la API key IMECF.
- Cambia la contraseña inicial del administrador.
- No uses AHG_DEMO_MODE=1 en producción.
- Usa AHG_AUTH_SECURE_COOKIE=1 en producción.
- Revisa el log de Vercel después de cada cambio de esquema.
- Para SQLite local conserva data/ahg_demo.db junto con su clave maestra.

13. Solucion de problemas

- No module named cryptography: ejecuta `pip install -r requirements.txt`.
- Define AHG_CREDENTIAL_SECRET: define la clave del paso 5 o la variable sensible en Vercel.
- **Error IPv6 al conectar Supabase:** usa Session pooler IPv4 y no el host directo db.<ref>.supabase.co.
- IMECF sin configurar: verifica IMECF_API_KEY, empresa, ambiente, validacion y conexion.

- **La preorden no aparece:** confirma que Vercel y el POS usan la misma DATABASE_URL de Supabase.
- **Vercel responde 500:** ejecuta `vercel.cmd logs` y revisa tipos booleanos, credenciales y migraciones.
- `gcloud` no se reconoce: instala Google Cloud CLI solo si necesitas administrar IMECF o servicios de GCP.

14. Checklist de entrega

- Repositorio descargado y dependencias instaladas.
- SQLite local probado.
- Proyecto Supabase creado y con contraseña disponible.
- DATABASE_URL usa Session pooler IPv4.
- Esquema PostgreSQL creado.
- Variables sensibles configuradas en Vercel.
- POS publicado en producción.
- Portal cliente, IA y preorden probados.
- Login administrativo probado.
- IMECF validado en ambiente de prueba.
- Tracking revisado.
- Contraseña inicial cambiada.
- Copias de seguridad y accesos restringidos.